

Structure of Project and State Management

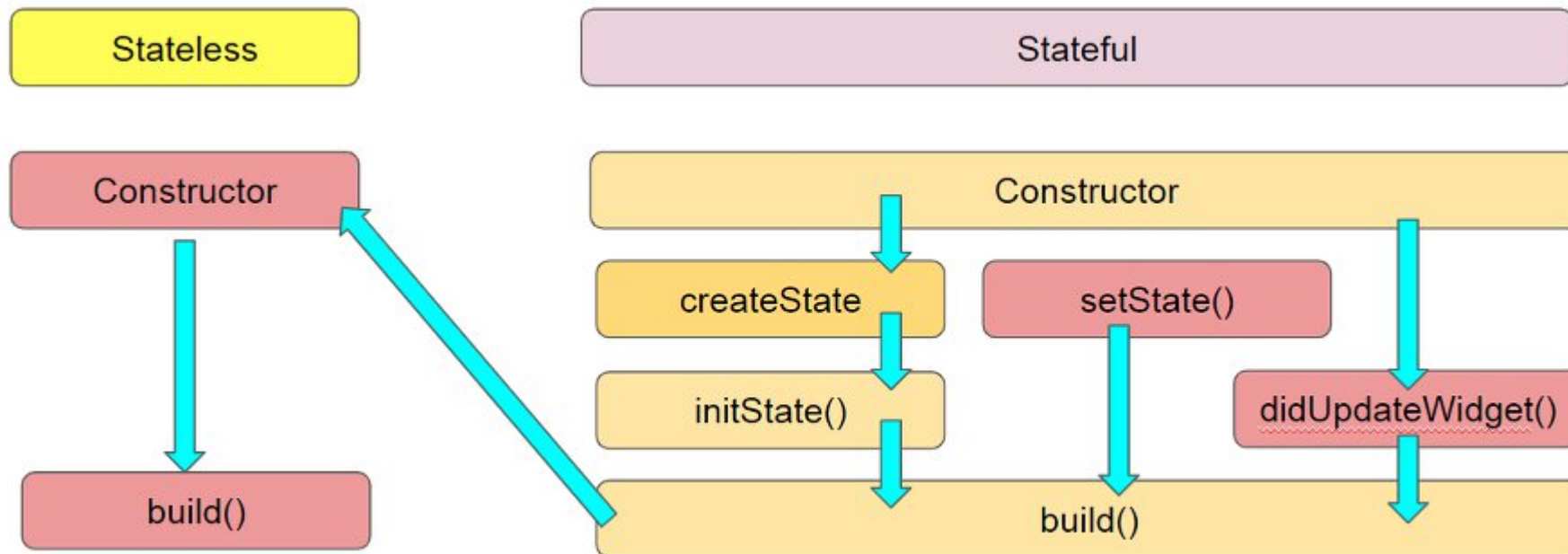
By Prasertsak U., Ph.D

Outlines

- StatefulWidget Life Cycle
- Structure of Project
- Business Logic
- โค้ดอย่างง่ายที่ไม่ได้ใช้ State Management
- รูปแบบการแชร์ความสามารถของโค้ดด้วยการใช้ *with* (Mixin)
- ตัวอย่างการใช้งาน State Management ด้วย *Provider* Package

StatefulWidget Life Cycle

- StatefulWidget นั้นเหมาะกับหน้าที่มีการเปลี่ยนแปลงอยู่ตลอดเวลาเพราะมี State ที่จะสามารถจัดการข้อมูลที่มีการเปลี่ยนแปลงได้ และจะสั่ง build ใหม่ด้วยตัวเอง



StatefulWidget Life Cycle

- ใน StatefulWidget ของ Flutter เมธอดที่ทำงานในตอนเริ่มต้นคือ **initState()** และตอนจบคือ **dispose()** โดยเมธอดเหล่านี้เป็นส่วนหนึ่งของ State Life cycle ที่ช่วยให้จัดการทรัพยากรได้อย่างถูกต้อง

- เมธอดตอนเริ่มต้น: **initState()** จะทำงานเพียงครั้งเดียวเมื่อ Widget ถูกสร้างขึ้น และนำไปใส่ใน Widget Tree เหมาะสำหรับ
 - การกำหนดค่าเริ่มต้นให้กับตัวแปร
 - การเรียกใช้ API หรือดึงข้อมูลจากฐานข้อมูล (Fetch Data)
 - การทำงานกับตัวจัดการ Listener ต่างๆ เป็นต้น

```
@override
void initState() {
  super.initState();
  // ทำการดึงข้อมูล หรือ กำหนดค่าเริ่มต้นที่นี่
}
```

- เมธอดตอนจบ: **dispose()** จะทำงานเมื่อ Widget ถูกลบออกจาก Widget Tree อย่างถาวร เหมาะสำหรับ
 - การปิดการทำงาน Listener ต่างๆ เพื่อป้องกันหน่วยความจำรั่วไหล
 - การยกเลิกการเชื่อมต่อต่าง ๆ
 - การทำลายคอนโทรลเลอร์ต่างๆ เช่น `animationController.dispose()` เป็นต้น

```
@override
void dispose() {
  // ทำลาย Controller หรือปิดการเชื่อมต่อที่นี่
  super.dispose();
}
```

Structure of Project สำคัญอย่างไร?

- การวางโครงสร้างโปรเจกต์ (Project Structure) เปรียบเสมือนการวาง **"ผังเมือง"** ก่อนที่จะสร้างสิ่งปลูกสร้าง ถ้าวางผังไม่ดี การขยายตัวในอนาคตจะทำได้ยาก แต่ถ้าผังเมืองดี ทุกอย่างจะชัดเจน ดูแลง่าย ซ่อมแซมแก้ไข รวมถึงการขยับขยาย จะทำได้ง่าย
- Debugging & Testing – การแก้ไขปัญหาเมื่อเกิดข้อผิดพลาดทำได้รวดเร็ว
- การทำงานร่วมกันเป็นทีม (Maintainability & Collaboration) - การวางโครงสร้างที่ชัดเจนเป็นส่วน จะช่วยลดระยะเวลาการอธิบายภายในทีมได้อย่างมาก เพราะภายในทีมจะเข้าใจตรงกัน และยังสามารถลดโอกาสในการเกิด conflict ในการแก้ไขโค้ดได้มากอีกด้วย (Git Conflict)

Structure of Project – โครงสร้างตัวอย่าง

- ตัวอย่างการจัดวางโครงสร้างสำหรับโปรเจกต์ขนาดเล็กถึงกลาง เพื่อให้ง่ายต่อการทำงาน เช่น

```
lib/  
├ main.dart      # ไฟล์หลักสำหรับเริ่มการทำงาน  
├ models/        # โฟลเดอร์เก็บ Class ที่เป็นโครงสร้างของข้อมูล (เช่น User, Product)  
├ pages/         # โฟลเดอร์เก็บ UI แยกเป็นหน้าต่าง ๆ  
├ services/      # โฟลเดอร์เก็บโค้ดที่ใช้ติดต่อกับ Hardware/Database (เช่น LocalStorageService)  
├ providers/     # (หรือ controllers) โฟลเดอร์เก็บ Business Logic ต่าง ๆ  
└ widgets/      # โฟลเดอร์เก็บ UI ขนาดเล็ก ที่มีการใช้ซ้ำในหน้า Pages ต่าง ๆ
```

Business Logic หมายถึงอะไร?

- **Business Logic** คือชุดของกฎและกระบวนการที่กำหนดวิธีการทำงาน เพื่อให้การตัดสินใจ และการทำงานเป็นไปอย่างมีเหตุผล และสอดคล้องกับเป้าหมาย
- ตัวอย่าง เช่น
 - **ระบบลงทะเบียนเรียน**: ถ้านักศึกษาลงวิชาใหม่ → ระบบต้องตรวจสอบว่ามีที่นั่งว่างหรือไม่ → ถ้าเต็มแล้วต้องแจ้งเตือน
 - **แอปธนาคาร**: ถ้าผู้ใช้โอนเงินเกินวงเงินที่กำหนด → ระบบต้องปฏิเสธและแจ้งเตือน

ระบบธนาคาร

- UI: ผู้ใช้กรอกจำนวนเงินที่จะโอน
- Business Logic: ตรวจสอบยอดเงินคงเหลือ, ตรวจสอบวงเงินสูงสุดต่อวัน, ถ้าเกินวงเงินให้ปฏิเสธ
- Database: บันทึกธุรกรรมการโอน

เนื้อหาจากคาบเรียนที่ผ่านมา (ต่อเนื่อง)

- ให้สร้างโปรเจกต์ใหม่ชื่อ **bodyhealth_app**
- สร้าง folder ชื่อว่า **model** และ **pages** ภายใน **/lib**
- สร้างไฟล์ชื่อ **bmr_pages.dart** ไว้ภายใน **pages**
- สร้างไฟล์ชื่อ **body_profile.dart** ไว้ภายใน **models** มีโครงสร้างประกอบด้วย
 - weight – **double**
 - height - **double**
 - age - **int**
 - gender - **String**

โครงสร้างไฟล์

```
/lib
- main.dart
- models/body_profile.dart
- pages/bmr_page.dart
```

รายละเอียดภายในไฟล์ body_profile.dart

```
1 class BodyProfile {
2   ·String gender;
3   ·int age;
4   ·double weight;
5   ·double height;
6
7   ·BodyProfile({
8     ··required this.gender,
9     ··required this.age,
10    ··required this.weight,
11    ··required this.height,
12    ··});
13 }
```

เนื้อหาจากคาบเรียนที่ผ่านมา (ต่อเนื่อง)

- สร้างแบบฟอร์มสำหรับกรอกข้อมูล ในไฟล์ **bmr_page.dart** ภายในโฟลเดอร์ **pages** เพื่อนำไปใช้ในการคำนวณ ค่า BMR โดยแบบฟอร์มประกอบด้วย น้ำหนัก ส่วนสูง อายุ เพศ (ชาย/หญิง) และมี 2 ปุ่มสำหรับกดเพื่อเริ่มคำนวณ และการ reset ค่า
- ให้นำข้อมูลที่กรอกเก็บลงในตัวแปรภายในคลาส **BodyProfile** ที่สร้างไว้
- ลักษณะหน้าจอที่ต้องการ ให้แสดงมีดังนี้

* ให้ทำการ Validate ความถูกต้องของข้อมูลให้ครบถ้วน

The image shows a mobile application interface for a BMR Calculator. The title bar is purple with the text 'BMR Calculator' in white. Below the title bar, there are four input sections: 1. Gender: A purple icon of two people is on the left. To the right are two radio buttons, 'Male' (selected) and 'Female'. 2. Weight (kg): A purple icon of a scale is on the left. To the right is a text input field containing '65'. 3. Height (cm): A purple icon of a person is on the left. To the right is a text input field containing '170'. 4. Age (years): A purple icon of a person is on the left. To the right is a horizontal slider ranging from 0 to 100, with a purple circle indicating the current value. At the bottom of the form are two rounded rectangular buttons: 'Calculate' and 'Reset'.

* ยังไม่ต้องทำการคำนวณ เพียงสร้างฟอร์มกรอกข้อมูลเท่านั้น

ต่อไปจะพัฒนาส่วนการคำนวณและแสดงผลลัพท์

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

Height is required.

Age:(years)

0 25 100

Calculate Reset

Please correct the errors in the form.

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

155

Age:(years)

0 21 100

Calculate Reset

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

155

Age:(years)

0 21 100

Calculate Reset

BMR Result

Your BMR is: 1202.75

ปิด

ตัวอย่าง APP

- จากโปรเจกต์ **bodyhealth_app** เดิมที่ได้จากเนื้อหาที่ผ่านมา
- สำหรับคนที่ไม่ได้สร้าง *bodyhealth_app* หรือไม่ได้ทำมาก่อนให้ดาวน์โหลดจาก [ที่นี่](#)
- ให้ปรับปรุงโครงสร้างโปรเจกต์ด้วยการสร้างโฟลเดอร์เพิ่มเติมดังนี้
- สร้างโฟลเดอร์ภายใน **/lib** ให้ประกอบด้วยโฟลเดอร์ต่อไปนี้

```
lib/  
├ main.dart      # ไฟล์หลักสำหรับเริ่มการทำงาน (เดิม-มีอยู่แล้ว)  
├ models/        # โฟลเดอร์เก็บ Class ที่เป็นโครงสร้างของข้อมูล (เดิม-มีอยู่แล้ว)  
├ pages/         # โฟลเดอร์เก็บ UI แยกเป็นหน้าต่าง ๆ (เดิม-มีอยู่แล้ว)  
└ providers/     # (หรือ controllers) โฟลเดอร์เก็บ Business Logic ต่าง ๆ (สร้างเพิ่มเติม)
```

สร้าง Code เพื่อรองรับ Logic ในการคำนวณค่า BMR

- สร้างไฟล์ชื่อ ***bmr_provider.dart*** ไว้ในโฟลเดอร์ **providers**
- เขียนโค้ดเพื่อคำนวณค่า BMR จากสูตรต่อไปนี้

ผู้ชาย: $BMR = 10W + 6.25H - 5A + 5$

ผู้หญิง: $BMR = 10W + 6.25H - 5A - 161$

หมายเหตุ

W = น้ำหนัก หน่วยเป็น กิโลกรัม

H = ความสูง หน่วยเป็น เซนติเมตร

A = อายุ หน่วยเป็น ปี

สร้าง Code เพื่อรองรับ Logic ในการคำนวณค่า BMR

bmr_provider.dart

```
class BmrProvider {  
  static double calculateBMR(  
    double weight,  
    double height,  
    int age,  
    String gender,  
  ) {  
    if (gender == 'Male') {  
      return (10 * weight) + (6.25 * height) - (5 * age) + 5;  
    } else if (gender == 'Female') {  
      return (10 * weight) + (6.25 * height) - (5 * age) - 161;  
    }  
    throw ArgumentError(  
      'Invalid gender. Please specify either "Male" or "Female".',  
    );  
  }  
}
```

การเขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ

- แก้ไขไฟล์ชื่อ **bmr_page.dart** ในโฟลเดอร์ **pages**
- การแก้ไขประกอบด้วย
 - การสร้างเมธอดหรือฟังก์ชันสำหรับเรียกใช้เมธอดการคำนวณใน providers
 - การตรวจสอบความถูกต้องของข้อมูลภายใน Form ก่อนการคำนวณ เนื่องจากผู้ใช้อาจกดปุ่มคำนวณทันทีโดยข้อมูลยังไม่ครบถ้วน
 - การเพิ่มโค้ดสำหรับการ reset ค่าภายใน Form
 - การเขียนโค้ดเพื่อแสดงผลลัพธ์ หลังจากการคำนวณค่า BMR เสร็จเรียบร้อยแล้ว โดยจะแสดงค่า BMR ที่คำนวณได้ ในลักษณะกล่องข้อความ

สร้างส่วนสำหรับแสดงผลการคำนวณ

- สร้างเมธอดชื่อ `_showAlertBox()` ไว้ในไฟล์ `bmr_page.dart`

```
void _showAlertBox(BuildContext context, {String title="info", required String message}) {  
  showDialog(  
    context: context,  
    barrierDismissible: false, // ผู้ใช้ต้องกดปุ่มเพื่อปิด AlertDialog  
    builder: (BuildContext context) {  
      return AlertDialog(  
        shape: RoundedRectangleBorder(  
          borderRadius: BorderRadius.circular(12.0),  
        ),  
        title: Row(  
          children: [  
            const Icon(Icons.info, color: Colors.blue),  
            const SizedBox(width: 8),  
            Text(title),  
          ],  
        ),  
        content: Text(  
          message,  
          style: const TextStyle(fontSize: 16),  
        ),  
        actions: [  
          ElevatedButton(  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
            child: const Text('ปิด'),  
          ),  
        ],  
      );  
    },  
  );  
}
```

เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ (cont.)

- เพิ่มเมธอดสำหรับการคำนวณ โดยสร้างเมธอดชื่อ ***doCalculation()***

```
void doCalculation() {  
    try {  
        double bmr = BmrProvider.calculateBMR(  
            bodyProfile.weight,  
            bodyProfile.height,  
            bodyProfile.age,  
            bodyProfile.gender,  
        );  
        // show result in a new page  
        _showAlertBox(context, title: "BMR Result", message: 'Your BMR is:  
        ${bmr.toStringAsFixed(2)}');  
    } catch (e) {  
        ScaffoldMessenger.of(context).showSnackBar(  
            SnackBar(  
                content: Text('Error calculating BMR: ${e.toString()}'),  
                backgroundColor: Colors.red,  
            ),  
        );  
    }  
}
```



เรียกใช้ logic การคำนวณใน providers



เปิดหน้า result เพื่อแสดงผล



แสดงแถบข้อความกรณีเกิดข้อผิดพลาด

เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ (cont.)

- เพิ่มโค้ดสำหรับตรวจสอบความถูกต้องของข้อมูลก่อนการคำนวณ ภายใต้ปุ่ม *Calculate*

```
Expanded(  
  child: OutlinedButton(  
    onPressed: () {  
      // Handle form submission  
      if (_formKey.currentState!.validate()) {  
        doCalculation();  
      } else {  
        ScaffoldMessenger.of(context).showSnackBar(  
          SnackBar(  
            content: Text(  
              'Please correct the errors in the form.',  
            ), // Text  
            backgroundColor: Colors.red,  
          ), // SnackBar  
        );  
      }  
    },  
    child: Text('Calculate'),  
  ), // OutlinedButton  
, // Expanded
```

ใช้คำสั่ง if เพื่อตรวจสอบความถูกต้องของข้อมูลทั้งหมดภายใน form หากมีข้อมูลใดที่ยังไม่ถูกต้องตามเงื่อนไขการ validate จะได้ค่าเป็น **false** กลับมา ซึ่งจะทำงานในส่วนของ else โดยจะแสดงแถบข้อความด้านล่างจอตด้วยพื้นหลังสีแดง แต่ถ้าข้อมูลถูกต้องจะได้ค่า **true** และจะเรียกใช้ **doCalculation()**



เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ (cont.)

- เพิ่มโค้ดสำหรับการล้างค่าหรือ reset ค่าภายใน Form ภายใต้อุปกรณ์ *Reset*

```
Expanded(  
  child: OutlinedButton(  
    onPressed: () {  
      // Handle form reset  
      _formKey.currentState?.reset();  
    },  
    child: Text('Reset'),  
  ), // OutlinedButton  
) // Expanded
```

คำสั่งสำหรับล้างค่าหรือ reset ค่าทั้งหมด
ภายใน Form

Calculate Reset

ทดสอบการทำงาน

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

Height is required.

Age:(years)

0 25 100

Calculate Reset

Please correct the errors in the form.

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

155

Age:(years)

0 21 100

Calculate Reset

BMR Calculator

Gender:

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

155

Age:(years)

0 21 100

Calculate Reset

BMR Result

Your BMR is: 1202.75

ปิด

ปัญหาชวนคิด..?!

- หากมีความต้องการใช้งานค่า BMR ในหน้าอื่นๆ อีกมากกว่า 1 จุด จะทำอย่างไร..?

จะต้องส่งข้อมูล BodyProfile ไปคำนวณใหม่ หรือจำค่าเดิมไว้ แล้วส่งไปแสดงยังจุดอื่น ๆ ดีหรือไม่ ?

การสร้าง Object ขึ้นมาใหม่ทุกครั้งที่ต้องการคำนวณ เช่น ถ้าสร้าง object ใน Page A เพื่อคำนวณ และต้องสร้างอีกตัวใน Page B ข้อมูลที่คำนวณไว้ใน Page A จะไม่ตามไปที่ Page B เพราะเป็นคนละก้อนกัน การสร้างใหม่เรื่อยๆ ทำให้แอปทำงานหนักขึ้นโดยไม่จำเป็น และเป็นการสิ้นเปลืองหน่วยความจำ

- หากต้องมีการแสดงค่า BMR ในเวลาเดียวกัน หลายจุดในหน้า Page จะต้องทำอย่างไรให้ค่าที่แสดงในแต่ละจุดอัปเดตเป็นค่าปัจจุบันอยู่เสมอ การใช้ setState() ถ้า App ขนาดเล็กก็ใช้ได้ แต่ถ้า App มีขนาดใหญ่ขึ้น การใช้ setState() ไปทุกแห่งที่มีการแสดงผลเพื่ออัปเดต UI นั้นค่อนข้างยุ่งยาก ไม่สวยงาม และทำให้โค้ดซับซ้อน แก้ไขได้ยากในอนาคต

แนวทางการแก้ไข...!

- ใช้คนกลางในการจัดการ State โดยยึดหลัก เมื่อ State เปลี่ยน → UI ต้องเปลี่ยนตาม
- วิธีการจัดการ State และการกระจาย State ไปยัง Widget ต่างๆ ทำอย่างไร ในที่นี่จะใช้ **provider package** เป็นตัวช่วย โดยจะสามารถเข้าถึงข้อมูลจากที่ไหนก็ได้
- widget ต่างๆ สามารถคอยฟัง Notify ได้ว่า state ของข้อมูลที่สนใจนั้น มีการเปลี่ยนแปลงหรือไม่ และสามารถทำการ Rebuild UI ได้ทันที หากข้อมูลที่กำลังสนใจนั้นมีการเปลี่ยนแปลง

ปัจจุบัน State Management Package มีให้เลือกใช้หลายตัวเลือก เช่น

- Provider
- Bloc
- Riverpod
- GetX

ลักษณะการเฝ้าฟัง Notify จาก Provider

- ลักษณะการคอยฟัง Notify ของ Provider แบ่งได้ 2 ลักษณะ ดังนี้

คำสั่ง	การใช้งาน	ใช้ตอนไหน
<code>context.read<T>()</code>	เข้าไปสั่งงาน หรือดึงค่า "ครั้งเดียว"	มักใช้ใน ฟังก์ชัน เช่น <code>onPressed</code>
<code>context.watch<T>()</code>	เฝ้าดูข้อมูล ถ้าข้อมูลเปลี่ยน ให้วาดหน้าจอใหม่	มักใช้ใน <code>build()</code> ของ Widget

T - Generic Type Parameter ในที่นี่จะถูกแทนที่ด้วยชื่อคลาสที่ต้องการใช้งาน

การใช้งาน Mixin (การผสมความสามารถ)

- การใช้งาน **with** (เช่น **with ChangeNotifier**) : การใช้กับคลาสจะมีแนวคิดที่แตกต่างจากการใช้ **extends** (การสืบทอดคลาส หรือ Inheritance)
- ความหมายและแนวคิดหลัก (Core Concept)
 - **extends** (Class Inheritance): เป็นการสร้างความสัมพันธ์ที่คลาสลูกจะสืบทอดคุณสมบัติ และพฤติกรรมทั้งหมดมาจากคลาสแม่ เหมาะกับการสร้างหมวดหมู่หรือลำดับชั้นของวัตถุ (Object Hierarchy)
 - **with** (Mixin): เป็นการนำชุดคำสั่ง หรือวิธีการทำงาน (Methods) มาบวกเพิ่มเข้าไปในคลาส โดยไม่ได้สร้างความสัมพันธ์แบบสายเลือด (ไม่ใช่แม่-ลูก) เปรียบเสมือนการใส่ความสามารถเสริม (Plug-in) ให้กับคลาสนั้น ๆ เท่านั้น

```
class Person with ChangeNotifier {  
  
}
```

การใช้งาน Mixin (cont.)

- ข้อแตกต่างที่สำคัญระหว่างการใช้ with กับ extends

คุณสมบัติ	extends (Inheritance)	with (Mixin)
จำนวนคลาสที่ใช้ได้	สืบทอดได้ เพียงคลาสเดียว (Single Inheritance)	ใส่ได้ หลายตัว พร้อมกัน (Multiple Mixins) โดยใช้คอมม่า คั่น เช่น with A, B, C
คอนสตรัคเตอร์ (Constructor)	คลาสแม่มีคอนสตรัคเตอร์ได้ และ คลาสลูกต้องเรียกใช้ผ่าน <code>super()</code>	Mixin ห้ามมี Generative Constructor (สร้างคอนสตรัคเตอร์รับค่าไม่ได้)
ความสัมพันธ์	เป็นสายเลือดเดียวกัน (Child เป็น Parent)	เป็นอิสระต่อกัน แค่มารับความสามารถไปใช้

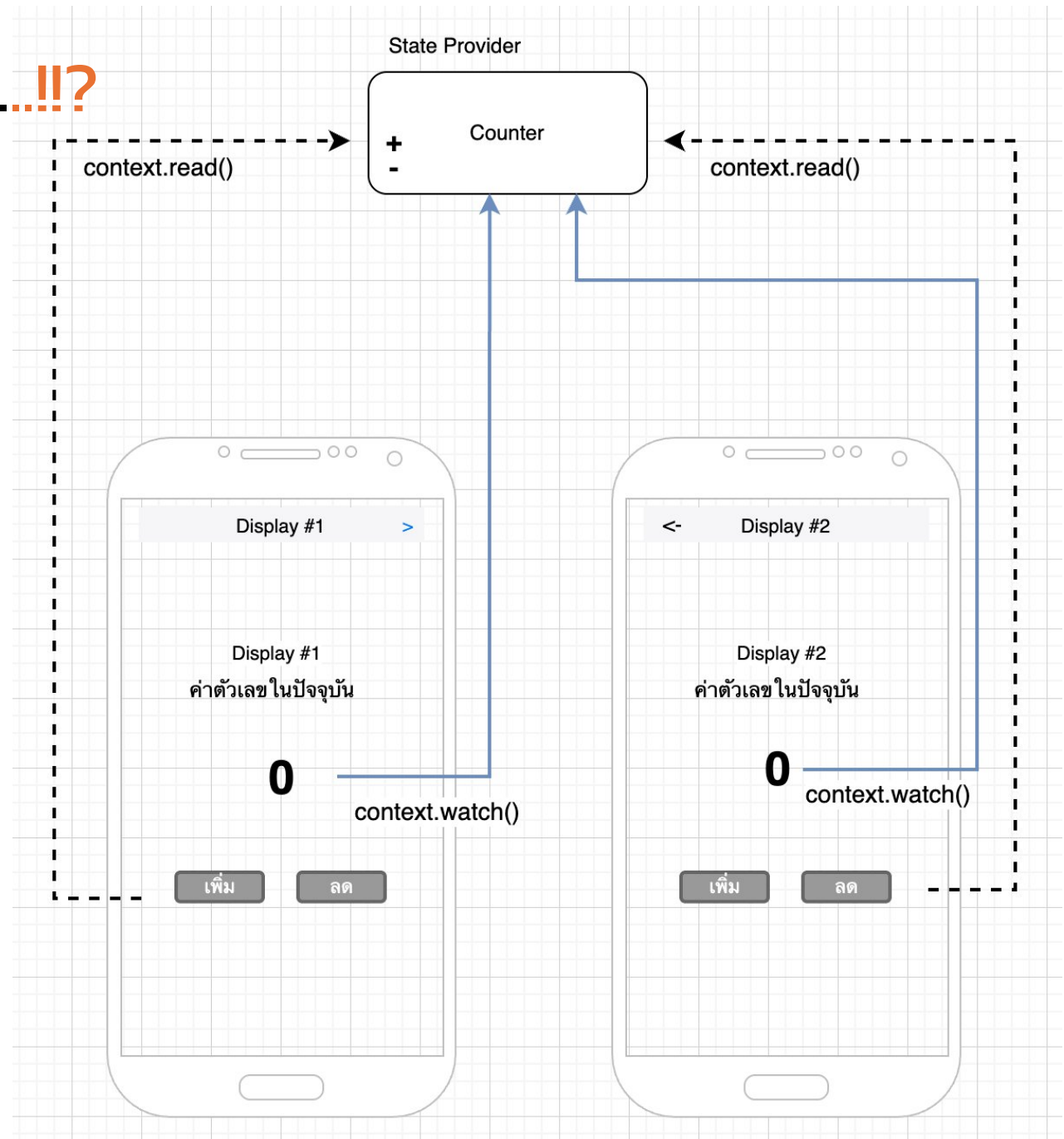
กฎข้อนี้ใช้กับ "ตัว Mixin" ไม่ใช่ "คลาสที่เอาไปใช้"



ตัวอย่าง App เพื่อความเข้าใจ...!!?

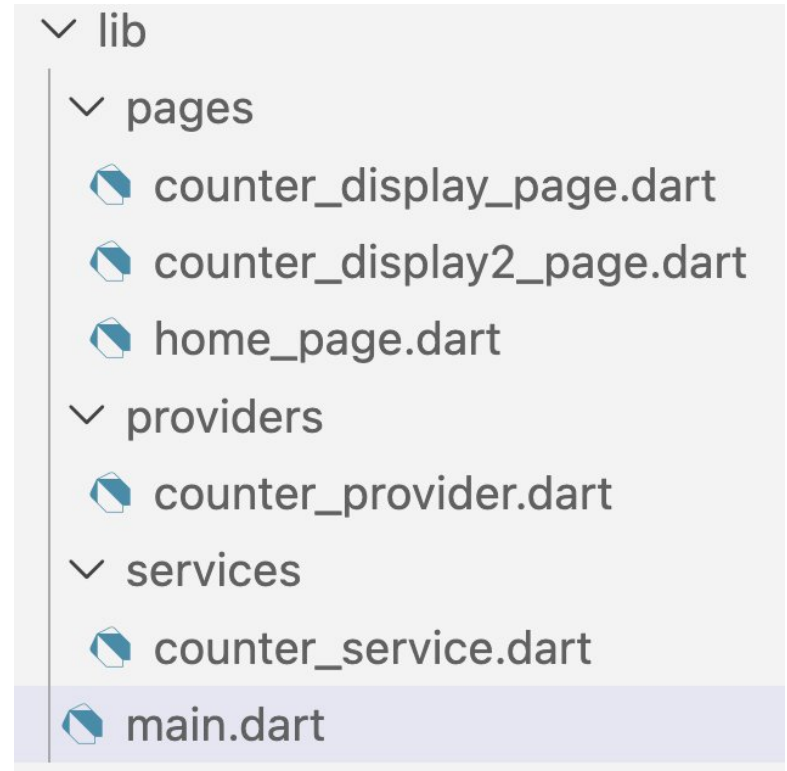
ตัวอย่าง App ที่มีการใช้งาน State Management Package และ Mixin

- ทำการจำลองการ เพิ่ม/ลด ค่าตัวเลข โดยจะแสดงค่าปัจจุบันของตัวเลขนั้นบนจอภาพ
- เมื่อกดปุ่มเพิ่มลด ไม่ว่าจะจากหน้าจอใดก็ตาม ค่าที่แสดงบน page: Display #1 หรือ Display #2 จะเป็นค่าปัจจุบันของตัวเลขอย่างถูกต้อง



ตัวอย่าง state_demo APP

- สร้างโปรเจกต์ใหม่ชื่อ **state_demo**
- ให้ดาวน์โหลดตัวอย่างโปรแกรมจาก ที่นี่
- นำไฟล์ทั้งหมดหลังจาก unzip ไปวางทับภายใน project ที่สร้างขึ้น
- ภายในจะประกอบด้วย code ภายในโฟลเดอร์ lib และไฟล์ pubspec.yaml



การใช้งาน State Management Package

- นำเข้า **provider package** มาใช้ในโปรเจกต์ ด้วยการเพิ่มเติมชื่อ **provider package** ไว้ในไฟล์ **pubspec.yaml**

pubspec.yaml

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino Icons font to your application.
35   # Use with the CupertinoIcons class for iOS style icons.
36   cupertino_icons: ^1.0.8
37   flutter_form_builder: ^10.3.0
38   form_builder_validators: ^11.3.0
39   provider: ^6.1.5
40
```

สร้าง Service แยกไว้ใช้งาน

- เขียนด้วย Business Logic ตามต้องการ เพื่อไว้เรียกใช้ในส่วนต่าง ๆ ของโปรแกรม

```
class CounterService {  
  
    int incrementValue(int currentVal) {  
        return currentVal + 1;  
    }  
  
    int decrementValue(int currentVal) {  
        return currentVal - 1;  
    }  
}
```

การสร้าง Provider โดยเพิ่มความสามารถของ State management

```
import 'package:flutter/material.dart';  
import '../services/counter_service.dart';
```

```
class CounterProvider with ChangeNotifier {
```

แปะความสามารถของ ChangeNotifier
เพิ่มเข้าไปด้วย with

```
  final CounterService _counterService = CounterService();
```

```
  int _counter = 0;
```

```
  int get counter => _counter;
```

ดึง service ที่เขียนแยกไว้มาใช้งาน
(วิธีดึง service แบบนี้อาจจะยังไม่สะดวก
ต่อการทำ Unit Test มากนัก)

```
  void increment() {  
    _counter = _counterService.incrementValue(_counter);  
    notifyListeners();  
  }
```

```
  void decrement() {  
    _counter = _counterService.decrementValue(_counter);  
    notifyListeners();  
  }
```

```
}
```

คำสั่ง **notifyListeners()** หากไม่มี ก็ไม่ทำให้โค้ดผิด เพียงแต่ widget
ที่มีการเฝ้าฟังสถานะของข้อมูลจะไม่ได้รับ notify ใด ๆ เมื่อค่า _counter
เกิดการเปลี่ยนแปลงจะส่งผลให้การแสดงผลไม่อัปเดตตามความเป็นจริง

ภายในส่วนของ page ต่าง ๆ สามารถเรียกใช้งาน Provider ได้

```
class CounterDisplay extends StatelessWidget {  
  const CounterDisplay({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    // ฟังค่าจาก Provider เพื่อแสดงผล  
    final counterProvider = context.watch<CounterProvider>();  
  
    return Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        const Text('Display Page #1'),  
        SizedBox(height: 20),  
        const Text('ค่าตัวเลขปัจจุบัน :'),  
        Text(  
          '${counterProvider.counter}',  
          style: const TextStyle(fontSize: 48, fontWeight: FontWeight.  
), // Text
```

เฝ้าฟังการเปลี่ยนแปลงจาก
CounterProvider

get ค่า counter มาแสดง

ภายในส่วนของ page ต่าง ๆ สามารถเรียกใช้งาน Provider ได้

```
24 Row(  
25   mainAxisAlignment: MainAxisAlignment.center,  
26   children: [  
27     ElevatedButton(  
28       onPressed: () => context.read<CounterProvider>().decrement(),  
29       child: const Text('ลด'),  
30     ), // ElevatedButton  
31     const SizedBox(width: 10),  
32     ElevatedButton(  
33       onPressed: () => context.read<CounterProvider>().increment(),  
34       child: const Text('เพิ่ม'),  
35     ), // ElevatedButton  
36   ],  
37 ), // Row
```

การเรียกใช้เมธอดใน provider เพื่อลดค่า

การเรียกใช้เมธอดใน provider เพื่อเพิ่มค่า

ภายใน main() จะต้องประกาศ Provider ด้วย

- แก้ไขไฟล์ **main.dart** ด้วยการประกาศส่วนของ Provider ที่จะใช้งาน

ให้เพิ่มการ import provider ไว้ในส่วนหัวด้วย: `import 'package:provider/provider.dart';`

```
void main() {  
  runApp(  
    MultiProvider(  
      providers: [  
        ChangeNotifierProvider(create: (_) => CounterProvider()),  
      ],  
      child: const MyApp(),  
    ),  
  );  
}
```

หากมีเพียง provider เดียวไม่จำเป็นต้องใช้แบบ MultiProvider ก็ได้ แต่ในที่นี้ใช้เพื่อไว้ให้เห็นว่า หากมีหลายตัวสามารถเพิ่มได้ใน []

Q/A